RBAC Task Manager

Spring Boot + DSA — Project Documentation

1. Project Overview

The **RBAC Task Manager** is a Spring Boot REST API that implements Role-Based Access Control (RBAC) for managing tasks across two roles — **Manager** and **Employee**. The project integrates several Data Structures & Algorithms (DSA) concepts directly into the service layer to achieve efficient in-memory caching, undo operations, priority retrieval, and advanced search/sort/group capabilities.

Technology Stack

Language	Java
Framework	Spring Boot, Spring MVC, Spring Security
Authentication	JWT (JSON Web Tokens) — Stateless
Persistence	Spring Data JPA + H2 In-Memory Database
DSA Used	Doubly Linked List + HashMap (LRU Cache), Deque + HashMap (Undo), Priority Queue (Max-Heap), HashMap / List (Sort & Group)
Exception Handling	Global via @RestControllerAdvice / @ExceptionHandler
Initial Data	@PostConstruct creates default manager (username: manager1, password: manager1) on first startup if no users exist

2. Roles & Entities

Roles

MANAGER	Creates tasks, assigns them to their own employees, can update/delete tasks they created, manages their own team of employees.

EMPLOYEE	Views tasks assigned to them, updates status/priority/name on their own tasks, can trigger undo on their last operations.

Entity: User

id	int — @Id @GeneratedValue (auto-incremented primary key)
userName	String — unique login username
password	String — stored as plain text (no BCrypt encoding in current implementation)
role	Enum — MANAGER | EMPLOYEE
manager	@ManyToOne — references the managing User (@JoinColumn name='manager')
employees	@OneToMany(mappedBy='manager') — @JsonIgnore — list of employees under this manager
assignedTasks	@OneToMany(mappedBy='assignedTo') — @JsonIgnore — tasks assigned to this user
createdTasks	@OneToMany(mappedBy='createdBy') — @JsonIgnore — tasks created by this user

Entity: Task

id	int — @Id @GeneratedValue
name	String — task title/description
priority	Enum — HIGH | NORMAL | LOW
status	Enum — PENDING | IN_PROGRESS | COMPLETED
assignedTo	@ManyToOne — User this task is assigned to (@JoinColumn name='assigned_to')
createdBy	@ManyToOne — User who created this task (@JoinColumn name='created_by')

3. URL Flow & Security Architecture

Every incoming HTTP request passes through the following stateless security pipeline:

Step	Component	What Happens
1	HTTP Request	Client sends request with JWT in Authorization header
2	Security Config	Determines if route is public (/auth/**) or protected

3	JWT Filter	Extracts token → validates using JwtUtil → sets Authentication in SecurityContext
4	Security Config (Authorization)	Checks role-based access. /manager/** → MANAGER only. /employee/** → MANAGER + EMPLOYEE. If unauthorized → HTTP 401
5	Controller	Routes request to the appropriate service method
6	Service Layer	Executes business logic. Reads/writes in-memory HashMaps (userTasksMap). LRU cache updated on specific GET operations.
7	Repository	DB queried only on first data load (loadData). Subsequent reads served from in-memory map.
8	Response	JSON response returned to client

Route Prefix → Role Mapping

/auth/**	Public — no authentication required (register, login)
/manager/**	MANAGER role only
/employee/**	Both MANAGER and EMPLOYEE roles

4. API Endpoints

4.1 Auth Endpoints

Endpoint	Access	Description
POST /auth/login	Public	Authenticate and receive JWT token. Returns JWT to be sent in Authorization header for all subsequent requests.

4.2 User Endpoints

Endpoint	Access	Description
GET /employee/getMe	MANAGER, EMPLOYEE	Get the currently authenticated user's own profile
GET /manager/get-all	MANAGER only	Get all employees managed by the calling manager. Returns manager itself if no employees exist yet.
POST /manager/create	MANAGER only	Create a new employee under the calling manager. Manager must be set correctly in request body.

DELETE /manager/delete/{dId}	MANAGER only	Delete an employee. Fails if employee manages others or has tasks assigned/created.

4.3 Task Endpoints

Endpoint	Access	Description
GET /employee/tasks	MANAGER, EMPLOYEE	Get all tasks. MANAGER: all tasks he created. EMPLOYEE: all tasks assigned to them. Loaded from in-memory map (DB on first load). Does NOT update LRU cache.
GET /employee/tasks/id/{id}	MANAGER, EMPLOYEE	Get task by ID from in-memory map. Updates LRU cache.
GET /employee/tasks/name/{name}	MANAGER, EMPLOYEE	Search tasks by name (case-insensitive, contains match). Updates LRU cache for each matching task.
GET /employee/tasks/highPriorityTask	MANAGER, EMPLOYEE	Returns highest-priority task using Priority Queue (lazy deletion). Updates LRU cache.
GET /employee/tasks/lru	MANAGER, EMPLOYEE	Returns the user's recently accessed tasks from LRU cache (display only). Most recently accessed shown first.
GET /employee/tasks/undo	MANAGER, EMPLOYEE	Undo the most recent operation (create/update/delete). Up to 5 operations stored.
POST /employee/tasks/sortByTasks	MANAGER, EMPLOYEE	Sort a given task list by: name, priority, status (supports multi-key sort via comma-separated values).
POST /employee/tasks/groupByTasks	MANAGER, EMPLOYEE	Group a given task list by: name, priority, or status. Returns Map<String, List<Task>>.
POST /manager/tasks/create	MANAGER only	Create a new task. Must assign to own employee. Cannot assign to other managers or their employees.
PUT /employee/tasks/update	MANAGER, EMPLOYEE	Update a task. See business rules for field-level restrictions per role.
DELETE /employee/tasks/delete/{id}	MANAGER only	Delete a task. Only the creating manager can delete. Actively removes from DB + in-memory map + LRU cache.

5. DSA Implementations

5.1 LRU Cache

Data Structure: Doubly Linked List + HashMap | **Limit:** 3 tasks per user

Purpose: The LRU cache is purely for **display** — it lets the user see their most recently accessed tasks via the **GET /employee/tasks/lru** endpoint. It is *not* used as a read-through cache before hitting the database.

How it works:

- The LRU cache is updated on: **getTaskById**, **getTasksByName**, **getPriorityTask**. **getAllTasks is excluded** — since all tasks are being fetched, there is no meaningful recency to track.
- When a task is accessed, it is moved to the **front** of the doubly linked list (most recently used position).
- When the cache exceeds the limit of 3, the node at the **tail** (least recently used) is evicted.
- On task **update**: the node's task reference is updated in-place (updateTaskLru) if it exists in cache.
- On task **delete**: the node is removed from both the linked list and the HashMap (deleteTaskLru). This is applied to both the manager's and employee's caches.
- The linked list uses sentinel **head** and **tail** dummy nodes to simplify insert/remove operations.

5.2 Undo Operations

Data Structure: Deque (LinkedList) + HashMap | **Limit:** 5 operations per user

Purpose: Undo reverses your **most recent operation** — not a specific task. Each press of undo reverses one operation going back in time, regardless of which task was involved.

How it works:

- Every **create**, **update**, or **delete** operation pushes an **UndoNode** (operation name + task snapshot) to the front of the user's Deque.
- Calling **undo** pops the front node and reverses that operation: *create* → *delete, delete* → *create, update* → *update back to old state*.
- Example: User performs task1 → task2 → task3 operations. First undo reverses task3 operation. Second undo reverses task2. Third undo reverses task1.
- **Cross-user invalidation:** If Manager has undo stack [task1, task2, task3] and task2 is assigned to emp2 who then updates task2, the manager's undo Deque is trimmed — all entries **from task2 onwards** are removed (task2 and task3). Manager's undo stack becomes [task1] only.
- This prevents a manager from undoing an operation that was superseded by another user's edit.
- The **userIsUndoOperationMap** flag prevents undo itself from being pushed onto the undo stack (undo of undo is blocked).
- If the Deque exceeds 5 entries, the oldest entry (tail) is automatically dropped.

5.3 Priority Queue — getPriorityTask

Data Structure: PriorityQueue (min-heap by Priority enum order, where HIGH < NORMAL < LOW in enum ordering)

How it works:

- Returns the **highest priority task** of the calling user.
- **Lazy deletion strategy:** When a task is deleted or updated, it is *not* immediately removed from the Priority Queue (costly operation). Instead, stale entries are cleaned up lazily at retrieval time.

- On getPriorityTask: peek the top of the queue. Validate the task still exists in the userTasksMap AND that the object reference matches (same instance). If invalid → poll and discard, check next. If valid → return task and update LRU.
- New tasks are added to the Priority Queue on both **create** and **update** (new version offered to queue).

5.4 Sorting & Grouping

Data Structure: Function<Task,String> and BiFunction<Task,Task,Integer> stored in HashMaps

How it works:

- The client sends a list of tasks (from a previous GET response) in the request body along with a sort/group key.
- **Sort** — supports **multi-key sorting** via comma-separated values (e.g. *priority,name*). Returns a sorted **List<Task>**. Tie-broken by task ID.
- **Group** — groups tasks by a single key (name, priority, or status). Returns **Map<String, List<Task>>**.
- Logic implemented entirely using **Function** and **BiFunction** lookups — **no if-else conditionals** in sorting or grouping logic.

6. Business Rules & Access Control

6.1 Task Creation

- Only **MANAGER** can create tasks.
- A manager can only assign tasks to employees directly under their management.
- Cannot assign to another manager or to another manager's employees.
- On creation: task is saved to DB, added to both manager's and employee's in-memory map, and offered to both Priority Queues.

6.2 Task Update

- **EMPLOYEE** can only update tasks in their own task map (assigned or created). Can edit: status, priority, name. Cannot change assignedTo.
- **MANAGER** can edit: status, priority, name, and **assignedTo** (reassign within their own team). Cannot reassign to other managers or their employees.
- Neither role can change the **createdBy** (owner) of a task.
- If a task is reassigned to a different employee, the old employee's task map and LRU cache are updated and their undo history for that task is cleared.

6.3 Task Deletion

- Only **MANAGER** can delete tasks (enforced via route prefix /employee/tasks/delete — accessible to manager too due to role hierarchy).
- A manager can only delete tasks they **created**.
- **Active deletion**: removed from DB, manager's map, employee's map, manager's LRU cache, and employee's LRU cache.
- Employee's undo history for that task is also cleared via deleteUndo.

6.4 getAllTasks & LRU Cache

- **EMPLOYEE** → gets all tasks assigned to them.

- **MANAGER** → gets all tasks they created.
- Data is loaded from DB into in-memory HashMap on **first call** only (loadData). Subsequent calls served entirely from memory.
- **getAllTasks does NOT update the LRU cache** — since all tasks are fetched, there is no single recently-accessed task to track.
- LRU cache is updated by: getTaskById, getTasksByName, getPriorityTask.

6.5 User Management

- Only MANAGER can create, list, or delete employees.
- A manager can only create employees assigned to themselves.
- An employee **cannot be deleted** if they manage other employees or have tasks assigned/created.
- On startup (@PostConstruct): if no users exist, a default manager is created (username: *manager1*, password: *manager1*).

7. Exception Handling

Endpoint	Access	Description
UnauthorizedUserException	403	Role violation — e.g. employee trying to create task, manager assigning to wrong employee
UserNotFoundException	404	User ID not found in repository
ContentNotFoundException	404	Task ID not found in user's task map
NoContentException	204	Result set is empty (no tasks found, empty undo stack, empty LRU cache)
BadRequestException	400	Invalid or missing sort/group feature parameter
InternalServerErrorException	500	Invalid undo operation type encountered

All exceptions are handled centrally via **@RestControllerAdvice** and **@ExceptionHandler**.

8. Database

Database: H2 In-Memory Database | **ORM:** Spring Data JPA (Hibernate)

- Tables auto-generated from JPA entity annotations.
- **users** table: id, user_name, password, role, manager (FK → users.id)
- **task** table: id, name, priority, status, assigned_to (FK → users.id), created_by (FK → users.id)
- Bidirectional relationship: User ↔ Task via @ManyToOne / @OneToMany.
- @JsonIgnore on collection fields prevents circular JSON serialization.
- DB is queried only on initial data load per user session. All subsequent reads/writes use in-memory HashMaps, with DB writes only on create/update/delete.

9. In-Memory Data Structures (TaskService)

userTasksMap	Map<Integer, Map<Integer,Task>> — primary in-memory store. Per-user task map (userId → taskId → Task).
userPriorityQueueMap	Map<Integer, PriorityQueue<Task>> — per-user max-priority heap. Lazy-deleted on retrieval.
userLruCacheTasksMap	Map<Integer, Map<Integer,LruNode>> — per-user LRU HashMap for O(1) node lookup.
userLruCacheHeadMap **userLruCacheTailMap**	Sentinel head/tail LruNodes per user for the doubly linked list.
userUndoDequeMap	Map<Integer, Deque<UndoNode>> — per-user undo stack (front = most recent).
userUndoTasksMap	Map<Integer, Map<Integer,Integer>> — tracks how many undo entries exist per taskId (for fast lookup/invalidation).
userIsUndoOperationMap	Map<Integer, Boolean> — flag to prevent undo operations from pushing to undo stack.

10. Feature Summary

Endpoint	Access	Description
LRU Cache	MANAGER, EMPLOYEE	Doubly Linked List + HashMap. Limit 3. Display-only (GET /lru). Updated by getById, getByName, getPriority. NOT updated by getAll.
Undo Operations	MANAGER, EMPLOYEE	Deque + HashMap. Limit 5. Undoes last operation in reverse chronological order. Cross-user edit invalidates affected undo entries.
Priority Queue	MANAGER, EMPLOYEE	PriorityQueue + lazy deletion. Validates against userTasksMap by reference equality before returning task.
Sort	MANAGER, EMPLOYEE	BiFunction map. Multi-key sort via comma-separated params. Returns List<Task>.
Group	MANAGER, EMPLOYEE	Function map. Single key group. Returns Map<String, List<Task>>. No if-else logic.
Create Task	MANAGER only	Own employees only. Saved to DB + both in-memory maps + both Priority Queues.

Delete Task	MANAGER only	Own created tasks only. Removed from DB + maps + LRU + employee undo stack.
Update Task	MANAGER, EMPLOYEE	Role-scoped. Employee: status/priority/name only. Manager: + reassign within own team.
JWT Auth	All	Stateless. Validated per request by JwtFilter via JwtUtil.
Exceptions	All	@RestControllerAdvice handles all errors centrally with appropriate HTTP status codes.